

Karachi AQI Predictor

Project Report: Forecasting Model Development,
Evaluation, and Deployment Architecture

End-to-end ML system for real-time and multi-horizon
Air Quality Index forecasting for Karachi, Pakistan

Primary metric	R2 Score (variance explained)
Forecast horizons	+1h • +24h • +48h • +72h
Final model	Horizon-specific XGBoost
Serving platform	Streamlit + Hopsworks Feature/Model Registry
Data sources	Open-Meteo (weather) • OpenAQ (PM2.5)

1. Project Overview

The Karachi AQI Predictor is an end-to-end machine learning system that monitors and forecasts the Air Quality Index (AQI) for Karachi, Pakistan. The system ingests hourly weather observations and PM2.5 pollution readings, engineers temporal features, stores processed data in a Hopsworks Feature Store, trains horizon-specific forecasting models, registers the selected models in the Hopsworks Model Registry, and serves live readings and forecasts through an interactive Streamlit dashboard.

The dashboard forecasts AQI at four horizons, matching the operational requirement of the project:

- **+1 hour** — immediate, near-term forecast
- **+24 hours** — next-day planning horizon
- **+48 hours** — two-day outlook
- **+72 hours** — three-day outlook

The primary evaluation metric throughout model development was the **R^2 score**, which measures the proportion of variance in true AQI values that a model's predictions explain. Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) were tracked as supporting metrics. An R^2 close to 1.0 indicates a near-perfect fit; an R^2 near 0 indicates the model performs about as well as predicting the historical mean AQI; a **negative** R^2 indicates the model performs worse than that naive mean-based benchmark.

1.1 What the Project Does

- Collects weather data for Karachi from Open-Meteo.
- Collects hourly PM2.5 data from OpenAQ.
- Cleans, merges, and validates the source data.
- Calculates AQI from PM2.5 measurements.
- Builds lag, rolling-window, change, percentage-change, and cyclical time features.
- Stores the training features in the Hopsworks Feature Store.
- Trains and evaluates several regression approaches.
- Registers optimized XGBoost models in the Hopsworks Model Registry.
- Reads the newest Hopsworks row inside the Streamlit application.
- Uses the registered models to generate live forecasts and explanations.

2. System Architecture

The system follows a linear pipeline from raw data collection through to dashboard-served inference. Weather data (Open-Meteo) and pollution data (OpenAQ) are independently collected, then cleaned and merged before feature engineering is applied.

```
Open-Meteo (weather) + OpenAQ (PM2.5)
|
v
```

```
Cleaning & Merging
|
v
Feature Engineering (lags, rolling stats, changes, cyclical encodings)
|
v
Hopworks Feature Store (aqi_training_features, v2)
|
v
Horizon-Specific Model Training (+1h / +24h / +48h / +72h)
|
v
Hopworks Model Registry
|
v
Streamlit Dashboard (live inference, alerts, explanations)
```

At inference time, the dashboard reads the latest feature-group history from Hopworks, reconstructs the engineered features required by each registered model, downloads the appropriate registry model, and performs inference using that model's own feature-name contract. This ensures the live serving path always matches the exact feature schema each model was trained on.

2.1 Hopworks Configuration

Resource	Value
Project	anaskaaqi
Feature group	aqi_training_features
Feature-group version	2
Feature view	aqi_72_hour_forecast
Model version	1 (default)

2.2 Registered Models

Forecast Horizon	Registry Model Name
+1 hour	aqi_1_hour_xgboost
+24 hours	aqi_24_hour_xgboost
+48 hours	aqi_48_hour_xgboost
+72 hours	aqi_72_hour_xgboost

3. Data Sources & Feature Engineering

The training data is keyed by hourly timestamps and combines original weather and pollution measurements with an extensive set of engineered temporal features. The dashboard intentionally displays only the original weather and pollution readings to end users; engineered features remain internal, since they exist purely as model inputs rather than user-facing measurements.

3.1 Engineered Feature Families

- **Lag features** — AQI values at 1, 2, 3, 6, 12, 18, 24, 36, 48, and 72 hours in the past.
- **Rolling statistics** — mean, standard deviation, minimum, and maximum over 3, 6, 12, 24, 36, 48, and 72-hour windows.
- **Change features** — AQI change and percentage-change over recent periods.
- **Cyclical encodings** — sine/cosine transforms of hour-of-day and day-of-week.
- **Calendar fields** — hour, day, month, year, and a weekend flag.

Lag and rolling features are generated *after* loading data from Hopsworks, rather than by modifying the Feature Store schema directly. This design choice allowed the team to experiment freely with temporal feature strategies without disrupting the production feature-store contract.

4. Model Development Journey

Model development progressed through several distinct stages, moving from a simple tree-based baseline to a fully optimized, horizon-specific gradient-boosting system, with a deep-learning architecture evaluated as a final comparison point.

4.1 Stage 1 — Random Forest Baseline

Random Forest was used as the initial baseline model. As an ensemble of independently trained decision trees whose predictions are averaged, it reduces the variance of individual trees and handles nonlinear, tabular relationships with minimal preprocessing. It established a benchmark against which more advanced models were compared, but was ultimately not selected because the project required stronger long-horizon forecasting performance.

4.2 Stage 2 — Transition to XGBoost

XGBoost (Extreme Gradient Boosting) builds trees sequentially, with each new tree correcting the errors of the existing ensemble, and uses a second-order loss approximation with explicit regularization to control complexity. This made it a strong candidate for AQI forecasting, offering strong tabular-data performance, nonlinear and feature-interaction modelling, overfitting control, GPU acceleration, and flexible hyperparameter tuning.

4.3 Stage 3 — Initial Multi-Horizon Model

The first major XGBoost experiment attempted to train a single strategy across the full 72-hour window. The results were poor across the board, with **negative R^2 values at every evaluated horizon** (roughly -0.70 to -0.86), indicating the model performed worse than a naive mean-based prediction. This motivated a change in strategy.

4.4 Stage 4 — Reduction to Four Required Horizons

Rather than modelling every hour from 1 to 72, the problem was simplified to the four horizons actually required by the application: +1h, +24h, +48h, and +72h. This reduced unnecessary computation and focused development directly on the required outputs.

4.5 Stage 5 — Horizon-Specific Models

A separate XGBoost model was trained for each horizon rather than forcing a single model to learn all forecasting distances at once — recognizing that predicting AQI 1 hour ahead is statistically a very different problem from predicting 48 or 72 hours ahead. This produced a substantial improvement in validation performance (+1h: 0.918, +24h: 0.689, +48h: 0.635, +72h: 0.643 validation R^2), confirming horizon-specific modelling as the superior strategy.

4.6 Stage 6 — Hyperparameter Optimization

An initial eight-configuration search identified a best validation setup (`max_depth=6`, `learning_rate=0.01`, `min_child_weight=5`, `subsample=0.95`, `colsample_bytree=0.95`, `reg_alpha=0.05`, `reg_lambda=2.0`), but its test-set performance dropped noticeably at longer horizons (average test R^2 of 0.524 vs. 0.689 on validation) — a clear illustration of the gap between validation and true held-out performance. The optimization strategy was subsequently refocused: the already-strong +1h model was retained, while additional search effort was concentrated specifically on +24h, +48h, and +72h, including a comparison between a full feature set and a reduced 'long-horizon' feature set with short-term features removed to limit overfitting.

4.7 Stage 7 — Deep-Learning Comparison

A PyTorch-based neural network was investigated to determine whether a deep-learning architecture could outperform the tuned XGBoost models. An initial training run suffered numerical instability (NaNs in the loss and predictions); after the pipeline was corrected, the model was successfully evaluated and narrowly outperformed XGBoost at +24h, but underperformed at +48h and +72h, giving it a lower overall average R^2 than XGBoost (approx. 0.609 vs. 0.624 in that comparison round).

5. Final Model Comparison

Four modelling approaches were carried through to final, horizon-aligned evaluation on the same four required forecast horizons. Only R^2 at +1h, +24h, +48h, and +72h is reported below, matching the dashboard's comparison view.

5.1 XGBoost (Horizon-Specific) — Final Production Model

Horizon	R2
+1h	0.8930
+24h	0.5605
+48h	0.5370
+72h	0.5283

XGBoost shows the desired and expected pattern: very high explanatory power at +1h (89.3%), followed by a controlled, gradual decline as forecast distance increases (56.1% → 53.7% → 52.8%). Crucially, it never drops into negative R^2 territory — every horizon still explains more than half of the AQI variance. This graceful degradation, rather than the abrupt collapse seen in the alternative models below, is the central reason XGBoost was selected as the production model.

5.2 Extra Trees Regressor

Horizon	MAE	R2
+1h	11.78	0.8837
+24h	26.23	0.5375
+48h	31.93	0.3702
+72h	27.88	0.5246

Extra Trees (Extremely Randomized Trees) is a bagged ensemble that, unlike Random Forest, selects split thresholds randomly rather than searching for the optimal split — typically trading a little extra bias for lower variance. It performs strongly at +1h (88.4%) and is competitive with XGBoost at +24h and +72h, but its accuracy dips sharply at +48h (37.0%), making it the least stable of the positive-scoring models across the four horizons.

5.3 MLP Regressor

Horizon	MAE	RMSE	R2
+1h	32.99	38.88	0.4872
+24h	50.92	60.76	-0.2295
+48h	50.44	61.59	-0.2977
+72h	43.86	53.90	-0.0223

The fully connected MLP Regressor explains a moderate 48.7% of variance at +1h — weaker than every tree-based model at that horizon, but still better than a naive mean guess. Beyond +1h, R^2 turns negative at all three longer horizons, with the weakest point at +48h. This reflects a common limitation of plain feed-forward networks on tabular time-series data: they can pick up some short-range signal from the most recent lag features but do not generalize well to longer-range temporal dependencies without explicit sequence modelling.

5.4 PyTorch Neural Network (Unstable Run)

Horizon	R2
+1h	-0.7731
+24h	-0.7006
+48h	-0.7667
+72h	-0.8005

This run corresponds to an early, pre-fix PyTorch training attempt, consistent with the numerical instability (NaN losses) noted during development before the training pipeline was corrected. Every horizon shows a negative R^2 , meaning the network's predictions were worse than simply predicting the mean AQI value at every horizon. Because performance does not improve even at +1h — the horizon every other model handles well — the failure pattern points to a training/convergence problem rather than a genuine horizon-difficulty effect, and this run should be read as a failed baseline rather than a fair representation of the corrected PyTorch approach.

5.5 Cross-Model Summary

Model	+1h R2	+24h R2	+48h R2	+72h R2
XGBoost	0.8930	0.5605	0.5370	0.5283
Extra Trees	0.8837	0.5375	0.3702	0.5246
MLP Regressor	0.4872	-0.2295	-0.2977	-0.0223
PyTorch (unstable)	-0.7731	-0.7006	-0.7667	-0.8005

XGBoost is the only model that remains positive and comparatively stable across all four required horizons — the basis for its selection as the production forecasting model. The alternative architectures are retained in the dashboard purely as documented comparison views, illustrating that greater model complexity (deep learning) does not automatically translate into better performance on this structured, tabular forecasting problem.

6. Final Model Selection Rationale

XGBoost was selected as the final production model for the following reasons:

- **Best overall performance** — highest and most consistent R^2 across all four horizons among the models compared.
- **Strong short-term forecasting** — R^2 of 0.893 at +1h demonstrates very strong immediate predictive capability.
- **Better long-horizon stability** — outperforms the alternative models at +48h and +72h, the primary challenge of the project.
- **Suitable for tabular environmental data** — the dataset is fundamentally structured/tabular, which gradient-boosted trees are specifically optimized for.
- **GPU compatibility** — supports GPU-based tree training, making extensive hyperparameter search practical in Google Colab.
- **Interpretability and deployment** — horizon-specific XGBoost models are comparatively straightforward to save, load, inspect (via native SHAP-compatible feature contributions), and deploy relative to a deep-learning architecture.

The project's overall conclusion is that the final result should be presented as a genuine test-set benchmark rather than claiming that an original target of $R^2 > 0.70$ was achieved. The system demonstrates strong short-term performance and useful, positive predictive performance across all four required forecasting horizons, while long-horizon forecasting remains the primary remaining challenge for future iterations.

7. Streamlit Dashboard

The dashboard application (`app/app.py`) is the primary user-facing component of the system, combining live readings, forecasts, and model transparency features.

- Latest raw weather and pollution readings in dark header cards.
- Dynamic daytime sun / nighttime moon icon on the temperature card.
- Current AQI gauge spanning six AQI ranges from 0 to 400.
- Color-coded AQI ranges: Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, and Hazardous.
- Hazardous and very-unhealthy alerts for current and forecast AQI values.
- Registry-backed predictions for +1h, +24h, +48h, and +72h.
- AQI and temperature trend charts with zoom, scroll-zoom, reset, and range-slider controls.
- Correlation heatmap across the displayed raw features and AQI.
- SHAP-compatible XGBoost feature contributions via native `pred_contribs`, with no separate SHAP installation required.
- Horizon buttons for viewing feature importance per registered model.
- A 'Model Applied' comparison section covering XGBoost, Extra Trees, MLP Regressor, and PyTorch.

- Model metrics, interpretation text, and model-selection rationale surfaced directly in the UI.

Only original weather and pollution features are shown to end users; engineered lag, rolling, change, and cyclical features remain internal to inference, since the registered models were trained with a larger feature contract than the raw values visible in the interface.

8. Project Structure & Deployment

8.1 Repository Layout

```
AQI_Predictor/
app/ Streamlit dashboard + requirements
data/ raw, historical, processed, current-year, ci_cd data
models/ local model artifacts and evaluation results
notebooks/ exploratory analysis and experiments
registry_uploads/ model-registry upload artifacts
src/
2026/ current-year ingestion and merge jobs
ci_cd/ automated data fetching and model registry jobs
data_collection/ API clients and collection configuration
data_processing/ cleaning and transformation logic
feature_Engineering/ feature groups, views, feature creation
models/ training, evaluation, and registration scripts
utils/ shared AQI utilities
tests/ test suite
requirements.txt
```

8.2 Requirements & Setup

- Python 3.11 or a compatible recent Python version.
- A Hopsworks account and API key.
- OpenAQ API access.
- Internet access for external data sources and Hopsworks.

Environment setup and launch commands:

```
python3 -m venv venv
source venv/bin/activate
pip install -r app/requirements.txt

streamlit run app/app.py
# default local URL: http://localhost:8501
# alternate port: streamlit run app/app.py --server.port 8502
```

Required environment variables (.env, loaded automatically via python-dotenv): HOPSWORKS_API_KEY, HOPSWORKS_PROJECT, OPENAQ_API_KEY, and optionally AQI_MODEL_VERSION.

8.3 Key Source Files

File	Purpose
app/app.py	Dashboard, Hopsworks reads, model downloads, live inference, alerts, charts, explanations

src/ci_cd/datafetch.py	Current data-fetching and Hopsworks update workflow
src/ci_cd/xgboost_model_registry.py	Feature preparation, training, and model-registry workflow
src/models/xgboost_model.py	XGBoost feature engineering and training logic
src/models/register.py	Local model upload and registration configuration
src/feature_Engineering/feature_group.py	Hopsworks feature-group setup
src/feature_Engineering/feature_view.py	Hopsworks feature-view setup
src/utlis/aqi.py	AQI calculation utilities

8.4 Security Notes

- Keep .env out of version control.
- Do not place API keys in README files or Python source files.
- Use separate development and production credentials where possible.
- Rotate credentials immediately if they are accidentally exposed.

9. Conclusion

The Karachi AQI Predictor demonstrates a complete, production-oriented machine learning workflow: from raw weather and pollution data collection, through feature engineering and a Hopsworks-backed feature store, to horizon-specific model training, rigorous model comparison, and live serving through a Streamlit dashboard. The modelling process progressed from a Random Forest baseline through increasingly sophisticated XGBoost configurations, temporal feature engineering, horizon-specific modelling, targeted hyperparameter optimization, and a final comparison against Extra Trees, an MLP Regressor, and a PyTorch deep-learning model.

The key finding across this process is that **increasing model complexity alone does not guarantee better forecasting performance**. The most effective improvements came from focusing on the four required forecasting horizons, training separate models per horizon, engineering informative lag and rolling features, and optimizing XGBoost specifically for the harder, longer-range horizons. The final XGBoost system offers strong short-term accuracy, positive and reasonably stable predictive performance across all four required horizons, and a deployment profile well suited to the project's Hopsworks-based feature and model registry architecture.